



SYSMON HUNTER

Detection Engineering & Analyst Console

User Manual

Version 0.3.3

Real-time Sysmon detection, correlation, and incident triage



Contents

1. Introduction

2. System Architecture

2.1 Event Pipeline

2.2 Data Flow

3. Detection Engine

3.1 Rule-Based Detection

3.2 Process-Tree Correlation

3.3 Statistical Beacon Detection

3.4 Reconnaissance-Burst Detection

3.5 Behavioral Baseline Detection

3.6 Ransomware Detection

3.7 Behavior Profiling & Derived Titles

3.8 Incident Scoring

4. Detection Rule Catalog

5. The Analyst Console

5.1 Incident Queue

5.2 Incident Detail Views

5.3 Explore View

5.4 Search

5.5 IOC Enrichment

5.6 ATT&CK Technique Reference

5.7 Analyst Notes

5.8 PDF Incident Reports

5.9 Incident Triage

5.10 Theme, Baseline Toggle & Database Reset

5.11 Live WebSocket Feed

6. REST API Reference

7. Configuration Reference

8. Installation & Getting Started

9. Testing

10. Docker Deployment

11. Design Decisions

12. Project Layout



1. Introduction

Sysmon Hunter is a real-time detection and correlation engine for Windows Sysmon telemetry, paired with a live analyst console. It ingests events from an endpoint, matches them against ATT&CK-mapped detection rules, reconstructs the process tree to correlate related detections into incidents, detects command-and-control beaoning and ransomware activity statistically, and enriches indicators against external reputation sources -- all streamed to the console in real time.

The project's guiding principle is that a single matched rule is a lead, not a verdict. A lone "PowerShell ran an encoded command" event is worth a glance. The same detection sitting under a Microsoft Word process, next to a reconnaissance burst and an outbound beacon, with a file hash confirmed malicious by VirusTotal, is an incident an analyst can act on immediately. Everything documented in this manual serves that one idea: turning individually weak signals into a correlated, prioritized, investigable incident.

This manual documents the detection engine, the analyst console, the REST and WebSocket API, the configuration surface, and the deployment options as of the version printed on the cover page.

Who this is for

Detection engineers writing or tuning Sysmon-based rules; SOC analysts triaging incidents in the console; and anyone standing up the engine against their own endpoint telemetry or a corpus of recorded Sysmon events.

2. System Architecture

2.1 Event Pipeline

Every event, regardless of source -- a live Winlogbeat feed, a replayed .evtx file, or a seed script -- takes the same path through the engine. The pipeline is independent of HTTP: it runs identically under the `/ingest` endpoint, the EVTXX replay script, and the automated test suite with no server running at all.

Step	Stage	What happens
1	Ingest	POST <code>/ingest</code> receives one normalized event and hands it to the pipeline. Holds no logic of its own.
2	Normalize	Winlogbeat/Sysmon JSON is converted into a uniform internal Event, so downstream code never branches on source format.
3	Observe	Every event feeds the in-memory process tree, not just the ones that match a rule -- a malicious process's ancestors are usually benign and must still be recorded.
4	Detect	The event is run against the YAML rule engine, the statistical beacon detector, the reconnaissance-burst detector, and -- when enabled -- the behavioral baseline detector.
5	Correlate	New detections are grouped with existing ones that share a process-tree root within the correlation window, forming or extending an incident.
6	Persist & Broadcast	The detection and incident are written to SQLite and pushed to every connected console over the WebSocket feed.



2.2 Data Flow

The process tree is keyed on Sysmon's ProcessGuid, never on process ID. Windows recycles PIDs aggressively, so a tree keyed on PID would graft an unrelated later process onto a malicious parent's identity the moment that PID is reused; ProcessGuid is unique across reboots and PID reuse and does not have this failure mode.

Incidents are not simply a rule match logged to a table -- the correlator groups every detection that shares a process-tree root within a configurable time window into a single incident, and carries the full branching tree with it, not just the linear chain from root to the triggering process.

3. Detection Engine

3.1 Rule-Based Detection

80 YAML detection rules, each mapped to one or more MITRE ATT&CK technique IDs, are indexed by Sysmon Event ID so that only relevant rules are evaluated per event. Rules use Sigma-compatible matching semantics: field/operator pairs (equals, contains, startswith, endswith, re) combined with an all/any condition, and an operator's expected value may be a single value or a list, OR'd together -- for example, matching a file write against any of several known ransomware extensions in one clause.

The full catalog, with severity and ATT&CK mapping for every rule, is in Section 4.

3.2 Process-Tree Correlation

Detections rarely stand alone in a real intrusion. The correlator reconstructs each host's process ancestry from Sysmon's ProcessGuid fields, and any detections whose processes share a tree root within the correlation window (10 minutes by default) are grouped into one incident carrying the complete branching tree -- not just the chain from root to the flagged process. A foothold that spawned several children shows every branch, whether or not each child individually fired a rule.

3.3 Statistical Beacon Detection

No single network-connection event can reveal a periodic command-and-control channel; it takes a sequence of them. The beacon detector watches outbound connections (Sysmon Event ID 3) per process and scores their timing for regularity, requiring a minimum number of callbacks before rendering a verdict. It uses the median and median absolute deviation of the intervals, not the mean and standard deviation -- a live C2 session produces occasional large outliers (the operator interacts, a callback retries late), and those outliers distort a standard-deviation score badly enough to lose the channel entirely, while the median barely moves. Jitter up to and beyond Cobalt Strike's default 37% is still caught. A configurable exclusion list keeps common legitimately-periodic processes (browsers, sync clients, chat apps) from generating noise.

3.4 Reconnaissance-Burst Detection

The discovery detector counts *distinct* ATT&CK discovery techniques observed in one process tree within a short window, not raw execution counts. A script re-running systeminfo in a loop is volume without variety and stays quiet; an operator running whoami, net, nltest, and systeminfo in quick succession is variety, and is flagged as a reconnaissance burst.



3.5 Behavioral Baseline Detection

Every detector above looks for a known bad pattern -- a rule matches a specific field shape, beaconing matches a specific statistical signature, a reconnaissance burst matches a specific breadth of recon commands. The baseline detector looks for the opposite: not whether something is bad, but whether it has ever been seen before on this host at all, which is what lets it catch a technique nobody has written a rule for yet. It learns, per host, which (process image, parent image) combinations are normal, and flags the first occurrence of a combination it has never recorded. The signal is deliberately weak on its own -- most first-time-seen processes are ordinary software updates or one-off admin tasks -- so it always reports at low severity and never alone drives a verdict; its role is to widen the net, not replace the rules that already cover well-understood techniques. A per-host learning phase (a configurable number of distinct combinations, unlogged and silent) keeps a fresh host's first stretch of ordinary activity from reading as a flood of anomalies, and what a host has learned persists across restarts independently of the detection and incident history, so an analyst clearing the database does not also reset a host's learned baseline. Off by default, this is the one detector an analyst can switch on or off live from the console's settings menu -- see §5.10 -- with no restart required.

3.6 Ransomware Detection

Two rules target the file-write behavior that precedes and accompanies a ransomware encryption event, in addition to the shadow-copy-deletion and recovery-disabling command-line rules that typically precede it: a ransom note dropped to disk, matched against the near-universal "how to decrypt / restore your files" naming convention used across ransomware families, and a mass file write ending in a known ransomware encryption extension (.locked, .encrypted, .crypt, .wcry, and others). Because these detections have no process command line to show as evidence -- the interesting fact is which file was written, not what process wrote it -- the console surfaces the exact matched file path everywhere the detection's detail is shown: the incident's detection list, its timeline and process-tree popups, and the full-screen tree viewer.

3.7 Behavior Profiling & Derived Titles

Rather than leaving an analyst to read a flat list of N detections, the profiling engine turns an incident's detections into a plain-language, kill-chain-ordered narrative -- for example: "gained initial access through a phishing document, executed an obfuscated PowerShell payload, harvested credentials from LSASS, beacons to C2 every ~35 seconds." Each incident is also given a derived title summarizing its contents at a glance in the queue -- "Phishing to reconnaissance," "Credential access with C2," "Ransomware preparation" -- instead of a generic "Incident #4821."

3.8 Incident Scoring

Incident severity scoring is non-linear by design: one critical LSASS access outweighs three medium-severity suspicious-path executions, and two high-severity detections together read as critical. This mirrors how an analyst actually judges risk -- a single strong signal can outweigh several weak ones, and enough weak signals together compound into a strong one. An incident is promoted to "active" once its cumulative score crosses a configurable threshold (12, by default).



4. Detection Rule Catalog

Every rule below was written and validated against real telemetry -- either a hand-built scenario exercised end-to-end through the console, or a known-malicious .evtx sample from a public detection-engineering corpus. Each has an automated true-positive case (the rule fires on the malicious sample) and a true-negative case (it stays quiet on adjacent legitimate activity) in the test suite.

Rule ID	Event	Severity	Title	ATT&CK
SYS-001	Process Create	High	Office application spawned a command interpreter	T1566.001, T1059
SYS-002	Process Create	High	PowerShell executed an encoded command	T1059.001, T1027
SYS-003	Process Create	High	LOLBin used to download a remote payload	T1105, T1218
SYS-004	Process Create	Critical	Volume shadow copies deleted	T1490
SYS-005	Process Create	High	Script engine spawned from a browser or mail client	T1204.002, T1566
SYS-006	Process Create	Medium	Executable ran from a user-writable staging path	T1204
SYS-007	Process Create	High	System binary running from the wrong path	T1036.005
SYS-008	Process Create	Medium	rundll32 executed without a DLL argument	T1218.011
SYS-009	Process Create	High	PowerShell download cradle	T1059.001, T1105
SYS-010	Process Access	Critical	Suspicious process accessed LSASS memory	T1003.001
SYS-020	Network Connection	Medium	Scripting or LOLBin process made an outbound connection	T1071
SYS-021	Network Connection	Medium	Connection to a common C2 port from a non-browser process	T1571
SYS-030	Registry Set	High	Autorun registry key modified	T1547.001
SYS-031	Registry Set	High	Windows Defender setting disabled via registry	T1685
SYS-032	Registry Set	High	Image File Execution Options debugger set	T1546.012
SYS-034	Process Create	High	Control panel item (.cpl) executed from a user-writable path	T1218.002
SYS-035	Process Create	High	Script host executing an encoded script	T1059.005, T1059.007, T1027
SYS-036	Process Create	High	Script host spawned by rundll32	T1059, T1218.011
SYS-037	Process Create	High	rundll32 invoking a known LOLBAS DLL export	T1218.011, T1204.001
SYS-038	Process Create	High	IIS credentials extracted via appcmd	T1552.001, T1003
SYS-039	Process Create	Medium	appcmd executed by a web server worker or script host	T1552.001
SYS-040	Image Load	High	PowerShell engine loaded by a non-PowerShell process	T1059.001
SYS-041	Process Access	Critical	LSASS opened with credential-dumping access rights	T1003.001
SYS-050	File Create	High	File dropped into a Startup folder	T1547.001
SYS-051	File Create	High	Office application wrote a script or executable	T1566.001
SYS-060	Pipe Created	High	Named pipe matching a known C2 default	T1071, T1055
SYS-070	Process Create	Medium	WMI persistence staged via mofcomp or wmic	T1546.003
SYS-071	WMI Event	High	WMI event consumer registered for code execution	T1546.003



Rule ID	Event	Severity	Title	ATT&CK
SYS-072	Pipe Created	High	Named pipe matching PsExec-style remote execution	T1021.002, T1569.002
SYS-073	Process Create	High	PsExec service binary executed on this host	T1569.002, T1021.002
SYS-074	Process Create	High	Regsvr32 registering a scriptlet via scrobj.dll (Squiblydoo)	T1218.010
SYS-075	Process Create	Medium	Certutil used to decode or encode a local file	T1140
SYS-076	Process Create	Medium	BITS job created via the Start-BitsTransfer cmdlet	T1197
SYS-077	Driver Load	High	Unsigned kernel driver loaded	T1211, T1562.001
SYS-078	Registry Set	High	Registry Run key written by the WMI provider host	T1047, T1547.001
SYS-079	Process Create	High	FTP client spawned a child process	T1105, T1202
SYS-080	File Create	Critical	Ransom note dropped	T1486, T1491.001
SYS-081	File Create	Critical	File written with a known ransomware encryption extension	T1486
SYS-082	Process Create	Critical	Credential dumping via comsvcs.dll MiniDump export	T1003.001
SYS-083	Registry Set	High	Fileless UAC bypass via auto-elevate registry hijack	T1548.002
SYS-084	Process Create	High	cmstp executed with a silent or auto-install flag	T1218.003
SYS-085	Registry Set	High	Port forwarding rule added via netsh	T1090.001
SYS-086	Registry Set	High	PowerShell script block logging disabled via registry	T1562.001
SYS-087	Registry Create/Delete	High	PowerShell Constrained Language Mode lockdown policy removed	T1562.001
SYS-088	Process Create	Critical	IIS worker process spawned a command shell	T1505.003
SYS-089	Process Create	Critical	SQL Server process spawned a command shell	T1059.003, T1190
SYS-090	Process Create	Medium	Process spawned by the PowerShell remoting host	T1021.006
SYS-091	Registry Set	Medium	Local account or administrators-group membership changed in the SAM hive	T1136.001, T1098
SYS-092	Process Create	High	PE metadata claims a trusted binary, but it runs from a staging path	T1036.005
SYS-093	Process Create	High	Scheduled task created via schtasks	T1053.005
SYS-094	Registry Set	High	New service image path points at a user-writable staging location	T1543.003
SYS-095	Remote Thread Created	Critical	Remote thread created inside LSASS	T1055, T1003.001
SYS-096	Process Create	High	Firewall rule added to allow inbound traffic	T1562.004
SYS-097	Process Create	High	Windows event log cleared via wevtutil	T1070.001
SYS-098	Process Create	Medium	Archive utility invoked with a password, staging data for exfiltration	T1560.001
SYS-099	Process Create	High	.NET installer utility executed from a user-writable staging path	T1218.004, T1218.009
SYS-100	Process Create	High	mshta executed a remote HTA	T1218.005
SYS-101	Process Create	Medium	Compiled HTML Help opened from a user-writable staging path	T1218.001



Rule ID	Event	Severity	Title	ATT&CK
SYS-102	Process Create	High	msiexec installed a package from a remote URL	T1218.007
SYS-103	Process Create	Medium	Account added to the local Administrators group via net.exe	T1098, T1136.001
SYS-104	Process Create	High	A security or update service was stopped from the command line	T1489, T1562.001
SYS-105	Process Create	High	Sysinternals sdelete executed	T1070.004
SYS-106	Process Create	Medium	cipher used to wipe free disk space	T1070.004
SYS-107	Registry Set	Medium	Remote Desktop enabled via registry	T1021.001
SYS-108	Process Create	Critical	Credential hive saved to disk via reg.exe	T1003.002
SYS-109	Process Create	Critical	Sysinternals procdump run against LSASS	T1003.001
SYS-110	Process Create	Critical	NTDS.dit extracted via ntdsutil	T1003.003
SYS-111	Process Create	High	mavinject used to inject into a running process	T1055.001
SYS-112	Process Create	Medium	WSL used to execute a command outside its Linux filesystem	T1202, T1059.004
SYS-113	Process Create	High	MSBuild ran a project file staged in a user-writable path	T1127.001
SYS-114	Process Create	Medium	Uncommon signed proxy-execution binary run against a staged target	T1218, T1216
SYS-115	Process Create	High	RDP session hijacked via tscon	T1563.002
SYS-116	Process Create	High	Active Directory reconnaissance tool executed	T1087, T1482
SYS-117	Process Create	Medium	Cloud sync/transfer tool executed	T1567.002
SYS-118	Process Create	High	Kerberoasting tooling executed	T1558.003
SYS-119	Process Create	High	PowerShell launched in version-2 downgrade mode	T1059.001
SYS-120	Process Create	High	Command line references a known AMSI-bypass signature	T1562.001
SYS-121	Process Create	Medium	bitsadmin used to transfer a file	T1197
SYS-122	Registry Set	High	CLSID InprocServer32 handler registered under a user's own hive	T1546.015
SYS-123	Process Create	High	Command line references a known privilege-escalation token-theft tool	T1134

Rule IDs follow no severity ordering; they are assigned sequentially as rules are added. "Event" is the Sysmon Event ID the rule is indexed under (Process Create = 1, Network Connection = 3, Driver Load = 6, Image Load = 7, Remote Thread Created = 8, Process Access = 10, File Create = 11, Registry Create/Delete = 12, Registry Set = 13, Pipe Created = 17, WMI Event = 20).



5. The Analyst Console

The console is a single-page, dark-surface application served directly by the backend at the site root, with a live WebSocket feed so every connected analyst sees the same state update in real time.

5.1 Incident Queue

The primary view: every incident the engine has correlated, most recent first, filterable between “Triage” (actionable incidents only, i.e. those over the score threshold), “All” (everything the engine is tracking, including sub-threshold activity it has not promoted), and “Closed” (incidents an analyst has already resolved -- see Section 5.9). Each row shows the derived title, severity, host, cumulative score, the process chain, and ATT&CK technique chips at a glance, before any drill-down.

5.2 Incident Detail Views

Expanding an incident reveals its behavior-profile narrative, then three interchangeable views of the same underlying data:

- **List** -- every detection with its full forensic context: process, parent, user, integrity level, hashes (with a one-click VirusTotal lookup), and, for file-write detections, the exact matched file path.
- **Timeline** -- detections placed in sequence with the real elapsed time between steps; clicking a node opens a floating forensic detail popup.
- **Process tree** -- the complete branching tree the incident spans, not just the chain to the triggering process. Nodes that fired a detection are colour-coded by severity; benign context processes are shown hollow so the whole tree's shape is visible, not only the flagged parts.

5.3 Explore View

The inline detail views are deliberately compact so they fit beside the detection list, which is too little room for a wide tree, a long timeline, or a long log list. “Explore” hands the same incident to a dedicated full-screen page (`/incident/{id}/explore`) with three tabs switchable in place, each a click away from the other: process tree, timeline, and a plain scrollable log. The tree and timeline gain click-and-drag panning, scroll-wheel zoom centred on the cursor, a zoom-to-fit control, and keyboard shortcuts (+ / - / 0). Clicking a node or a timeline entry opens the same forensic detail panel used inline.

5.4 Search

One search box combines free text with field filters, mixed freely in a single query -- for example, `powershell severity:critical host:fin-ws`. Free text matches anywhere an analyst might remember something about an incident: a process name, a command-line fragment, a hash, a rule ID, the title. Field filters narrow precisely by substring match, not exact match, so `host:fin-ws` finds `FIN-WS-07`.

Filter	Matches
<code>host:</code>	Substring match against the incident's host.
<code>severity:</code>	Exact match: info, low, medium, high, or critical.
<code>technique:</code>	Matches an ATT&CK technique ID, exact or as a prefix.
<code>rule:</code>	Matches a detection rule ID.
<code>user:</code>	Matches the forensic user field on a detection.



Filter	Matches
command_line:	Scoped substring match against a detection's command line only.
actionable:	true/false -- restricts to incidents over (or under) the score threshold.

5.5 IOC Enrichment

IP addresses, domains, and file hashes can be checked on demand against AbuseIPDB (IP reputation) and VirusTotal (IP, domain, and file-hash reputation). Results are cached for an hour. Every provider degrades gracefully to “unavailable” with no API key configured -- enrichment never blocks the rest of the console from working. Private IP ranges are never sent to a third party, and the strongest available hash (SHA-256 over SHA-1 over MD5) is always preferred, since MD5 collisions are cheap enough for malware authors to produce deliberately.

5.6 ATT&CK Technique Reference

Every technique chip shown anywhere in the console is clickable and opens its official MITRE description -- name, tactics, and full text -- from a local copy of the ATT&CK STIX dataset, with a link out to the ATT&CK site. No lookup leaves the console for an analyst who does not have T1003.001 memorized.

5.7 Analyst Notes

Each incident's full-page view carries a free-text analyst note, capped at 500 words -- a summary, not a report -- with autosave on the incident page and a live word counter.

5.8 PDF Incident Reports

Every incident can be downloaded as a self-contained PDF report, suitable for attaching to a ticket or handing to an incident-response lead. It is not a screenshot of the console -- it is the investigation written down: the behavior-profile narrative, the process chain, every detection with its forensics, and the incident's key indicators. Generated server-side with reportlab, requiring no headless browser in the deployment image.

5.9 Incident Triage

Every incident carries a status: open, closed, or a verdict such as false positive or benign, set from a “Set verdict” menu on the incident card and cleared with a “Close” action. Any state can transition to any other -- a closed incident can be reopened if new evidence surfaces -- and there is no workflow enforced beyond that; the analyst is the one doing the triage, not the console. Closed incidents drop out of the default queue view and are reachable from the “Closed” filter tab (Section 5.1), so a shift's queue only ever shows what still needs a decision.



5.10 Theme, Baseline Toggle & Database Reset

A settings menu in the console header holds a light/dark theme toggle, a live on/off switch for the behavioral baseline detector (§3.5), and one destructive action: wiping the database. The baseline switch is the one setting in this project that changes without a restart -- it writes through to the database and takes effect on the very next event, and every connected console reflects the change immediately over the same WebSocket feed used for live updates, not just the tab that clicked it. The reset is confirmed with a dialog, since there is no undo -- every detection and incident is permanently removed, on every connected console, via that same broadcast mechanism. Note that a database reset does not erase a host's learned baseline -- see §3.5.

5.11 Live WebSocket Feed

The console holds a persistent WebSocket connection and reconnects automatically on drop, so it can be left open on a wall display indefinitely. New detections, incident updates, and database resets are pushed to every connected client the instant they happen -- there is no polling and no manual refresh.



6. REST API Reference

The full interactive OpenAPI documentation is served at `/api/docs` (Swagger UI) and `/api/redoc` (ReDoc) on a running instance. The table below is a static summary of every route.

Method	Path	Description
POST	<code>/ingest</code>	Accept one normalized Sysmon/Winlogbeat event; runs it through the full pipeline.
GET	<code>/incidents</code>	List incidents, most recent first.
GET	<code>/incidents/{id}</code>	Full incident detail: detections, process tree, forensics.
PUT	<code>/incidents/{id}/status</code>	Close an incident, mark it a false positive, or reopen it.
PUT	<code>/incidents/{id}/notes</code>	Set the incident's analyst note (500-word limit).
DELETE	<code>/incidents/{id}/notes</code>	Clear the incident's analyst note.
GET	<code>/incidents/{id}/profile</code>	Kill-chain narrative summary for the incident.
GET	<code>/incidents/{id}/report</code>	Generate and download the incident's PDF report.
GET	<code>/detections</code>	List raw detections, most recent first.
GET	<code>/search</code>	Free-text and field-filtered search across incidents.
GET	<code>/attack/{technique_id}</code>	MITRE ATT&CK technique description, from the local dataset.
GET	<code>/enrich</code>	On-demand reputation lookup for an IP, domain, or hash.
DELETE	<code>/admin/database</code>	Wipe all detections and incidents; reset the live engine.
GET	<code>/admin/settings</code>	Current value of every runtime-editable setting (e.g. the baseline toggle).
PUT	<code>/admin/settings</code>	Flip a runtime setting; effective immediately, broadcast to every console.
GET	<code>/health</code>	Liveness/readiness probe: rule count, tracked processes.
WS	<code>/ws</code>	Live event stream: new detections, incident updates, resets.
GET	<code>/</code>	The analyst console (single page app).
GET	<code>/incident/{id}</code>	Full-page view of one incident, with notes editor.
GET	<code>/incident/{id}/explore</code>	Full-screen Explore view: process tree, timeline, or logs, picked by tab.
GET	<code>/incident/{id}/tree</code>	Alias for <code>/explore?view=tree</code> , kept for old links.



7. Configuration Reference

Every tunable lives in one settings object and can be overridden from the environment or a `.env` file, without touching code. All variables are prefixed `HUNTER_`.

Variable	Default	Purpose
HUNTER_DB_URL	sqlite+aiosqlite:///data/hunter.db	Database connection string.
HUNTER_CORRELATION_WINDOW_MINUTES	10	Time window for grouping detections into one incident.
HUNTER_INCIDENT_SCORE_THRESHOLD	12	Cumulative severity score to promote an incident to active.
HUNTER_PROCESS_TTL_MINUTES	120	How long an inactive process stays in the in-memory tree.
HUNTER_BEACON_ENABLED	true	Enable/disable statistical beacon detection.
HUNTER_BEACON_MIN_CONNECTIONS	6	Callbacks required before a beacon verdict.
HUNTER_BEACON_REGULARITY_THRESHOLD	0.75	Minimum regularity score (0-1) to flag a beacon.
HUNTER_BEACON_MIN_INTERVAL_SECONDS	5.0	Fastest interval considered beacon-like, not streaming.
HUNTER_BEACON_MAX_INTERVAL_SECONDS	3600.0	Slowest interval still distinguishable from a scheduled task.
HUNTER_DISCOVERY_ENABLED	true	Enable/disable reconnaissance-burst detection.
HUNTER_DISCOVERY_MIN_DISTINCT	4	Distinct recon techniques required to flag a burst.
HUNTER_BEHAVIOR_BASELINE_ENABLED	false	Fallback default for the baseline detector; overridden by the live console toggle once set.
HUNTER_BASELINE_LEARNING_EVENTS	50	Distinct combinations a host must contribute before the detector starts alerting.
HUNTER_ABUSEIPDB_API_KEY	(unset)	Optional key for IP reputation enrichment.
HUNTER_VIRUSTOTAL_API_KEY	(unset)	Optional key for hash/IP/domain reputation enrichment.
HUNTER_ENRICHMENT_CACHE_TTL_SECONDS	3600	How long a reputation lookup is cached.



8. Installation & Getting Started

Requires Python 3.11 or newer.

```
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt

python -m alembic upgrade head    # create the database schema
python -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

Console: <http://localhost:8000> · API docs: <http://localhost:8000/api/docs>

Seeding demo data

```
python scripts/seed_apr.py    # one deep multi-stage intrusion
python scripts/seed_demo.py    # varied incidents across the kill chain
python scripts/seed_rw.py    # a full ransomware chain in one incident
```

Analysing a real sample

Replay a Sysmon .evtx file -- from a lab VM, or a public corpus such as EVT-X-ATTACK-SAMPLES, where each file exercises one ATT&CK technique:

```
pip install evtx
python scripts/replay_evtx.py --file samples/sysmon_credential_access.evtx
```

Optional: IOC enrichment API keys

Works with no keys configured (every provider reports unavailable). For live reputation data, add free-tier API keys to a .env file:

```
HUNTER_ABUSEIPDB_API_KEY=...
HUNTER_VIRUSTOTAL_API_KEY=...
```

9. Testing

```
pip install pytest pytest-asyncio
python -m pytest    # 357 tests
```

The test suite doubles as documentation: each design decision named in Section 11 has a test named after it, and every shipped detection rule is validated against a true positive it must catch and a true negative it must ignore.

10. Docker Deployment

```
docker compose up --build
```

A multi-stage build keeps the runtime image to the application and its virtual environment only -- the build tooling is discarded. The container runs as an unprivileged user, applies database migrations automatically on start, and exposes a health check at /health.

11. Design Decisions

The choices below are what separate this engine from a pattern match over a log file. Each is enforced by an automated test named after it.

**Correlation keys on ProcessGuid, never PID.**

Windows recycles process IDs aggressively; a PID-keyed tree would graft a malicious child onto whatever unrelated process later inherits its parent's PID. ProcessGuid is unique across reboots and PID reuse.

The process tree observes everything, not just detections.

A malicious process's ancestors are usually entirely benign, so the process that never triggers a rule on its own must still be recorded, or it can never appear as the root of a phishing chain.

Incident scoring is non-linear.

One critical LSASS access outweighs three medium-severity suspicious-path executions. Two high-severity detections read as critical together. An incident can be worse than any single rule that composes it.

Beaconing uses median and median absolute deviation, not mean and standard deviation.

A live C2 session produces outliers constantly. One unusually long gap in an otherwise 60-second beacon wrecks a standard-deviation-based score and loses the channel; the median barely notices. Jitter up to Cobalt Strike's default 37% is still caught.

Discovery counts distinct techniques, not raw execution counts.

A script re-running one command in a loop is volume without variety. An operator running several different reconnaissance commands in quick succession is variety, and only the second pattern is flagged.

The baseline detector's learning phase is scoped per host and persisted, not per session.

Gating on distinct combinations a host has already contributed, loaded from the database at startup, means a host with a mature baseline keeps alerting immediately after a restart, while a host genuinely new to the detector still gets a quiet learning period -- either way, no cold-start alert storm.

Enrichment works with no API keys, and never leaks internal data.

Every provider degrades to "unavailable" without a key rather than failing the request. Private IP ranges are never sent to a third party. The strongest available hash is always preferred over a weaker one.

12. Project Layout

```
sysmon-hunter/
+-- backend/
|   +-- main.py          FastAPI app, lifespan, background sweep
|   +-- config.py        all tunables
|   +-- api/             ingest, detections, incidents, attack, enrich,
|   |                   report, search, notes, admin, ws, serializers
|   +-- engine/          normalizer, rule_loader, matcher, correlator,
|   |                   beacon, discovery, attack, enrichment, search,
|   |                   report, profile, pipeline
|   +-- models/          schemas, db
|   +-- data/            attack_data.json (ATT&CK technique lookup)
+-- rules/              80 YAML detection rules, by Event ID
+-- frontend/           console.html, incident.html, tree.html, static/{css,js}
+-- migrations/         Alembic
+-- scripts/            seed_apr, seed_demo, seed_rw, seed_full_coverage,
|                       replay_evtx, fetch_attack
+-- docs/               screenshots
+-- tests/              357 tests
```



Built as a hands-on threat-detection research project. The engine targets a single collector; scaling to many would mean moving the queue to Redis and the store to Postgres, both isolated behind small interfaces so that change touches one file each.